



DÉVELOPPER AVEC EXPRESS



CRÉER UN NOUVEAU PROJET EXPRESS

- **Étape 1 : créer le dossier « app-express »**

- **Étape 2 : initialiser npm**
npm init -y

- **Étape 3 : installer Express**

npm install express

L'installation d'Express se fait avec npm dans le dossier du projet.

- **Étape 4 : installer nodemon**

npm install nodemon --save-dev

nodemon permet de relancer automatiquement le serveur quand on modifie le code.

Note ne pas oublier de mettre dans package.

2. Structure de départ du projet

Pour commencer :

```
module-4-express/  
|  
├─ app.js  
├─ package.json  
└─ node_modules/
```

Créer un fichier :

```
app.js
```

```
1  {  
2    "name": "app-express",  
3    "version": "1.0.0",  
4    "description": "",  
5    "main": "app.js",  
6    "scripts": {  
7      "test": "echo \\\"Error: no test specified\\\" && exit 1",  
8      "start": "node app.js",  
9      "dev": "nodemon app.js"  
10   },  
11   "keywords": [],  
12   "author": "",  
13   "license": "ISC",  
14   "type": "module",  
15   "dependencies": {  
16     "express": "^5.2.1"  
17   },  
18   "devDependencies": {  
19     "nodemon": "^3.1.14"  
20   }  
21 }  
22
```



Fichier `app.js`

```
</> JavaScript

import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("Bonjour depuis Express");
});

app.listen(3000, () => {
  console.log("Serveur lancé sur http://localhost:3000");
});
```

Lancer le serveur

```
</> Bash

npm run dev
```

Ouvrir :

```
http://localhost:3000
```

Résultat :

```
Bonjour depuis Express
```

```
</> JavaScript
```

```
import express from "express";
```

On importe Express.

```
</> JavaScript
```

```
const app = express();
```

On crée une application Express.

```
</> JavaScript
```

```
app.get("/", (req, res) => {
  res.send("Bonjour depuis Express");
});
```

On crée une route en méthode `GET` .

Cette route répond à l'URL :

```
/
```

```
</> JavaScript
```

```
res.send("Bonjour depuis Express");
```

`res.send()` envoie une réponse au navigateur.

```
</> JavaScript
```

```
app.listen(3000, () => {
  console.log("Serveur lancé sur http://localhost:3000");
});
```

Le serveur écoute sur le port `3000` .



NOTION DE ROUTAGE

- Le routage consiste à dire : quand l'utilisateur demande telle URL, quelle fonction doit répondre ?

- Exemples :

```
/           → page d'accueil  
/about      → page à propos  
/contact    → page contact  
/products   → liste des produits
```

- Avec Express, on écrit :

```
app.get("/contact", (req, res) => {  
  res.send("Page contact");  
});
```

Fichier `app.js`

```
</> JavaScript  
  
import express from "express";  
  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("<h1>Accueil</h1><p>Bienvenue sur notre site Express.</p>");  
});  
  
app.get("/about", (req, res) => {  
  res.send("<h1>À propos</h1><p>Application créée avec Express.</p>");  
});  
  
app.get("/contact", (req, res) => {  
  res.send("<h1>Contact</h1><p>Email : contact@example.com</p>");  
});  
  
app.listen(3000, () => {  
  console.log("Serveur lancé sur http://localhost:3000");  
});
```

`http://localhost:3000`

`http://localhost:3000/about`

`http://localhost:3000/contact`



GESTION DES RÉPONSES

- Express permet d'envoyer plusieurs types de réponses.

Envoyer du texte

JavaScript

```
app.get("/message", (req, res) => {  
  res.send("Ceci est un message texte");  
});
```

Envoyer du HTML

JavaScript

```
app.get("/html", (req, res) => {  
  res.send("<h1>Titre HTML</h1><p>Paragraphe HTML</p>");  
});
```

Envoyer du JSON

JavaScript

```
app.get("/api/users", (req, res) => {  
  const users = [  
    { id: 1, name: "Amina" },  
    { id: 2, name: "Karim" }  
  ];  
  
  res.json(users);  
});
```

Ouvrir :

<http://localhost:3000/api/users>

Résultat :

JSON

```
[  
  { "id": 1, "name": "Amina" },  
  { "id": 2, "name": "Karim" }  
]
```

Avec Express, `res.json()` simplifie l'envoi de JSON.



PARAMÈTRES DE ROUTE

- Un paramètre de route est une valeur placée directement dans l'URL.
- Exemple : `/products/1`
- Ici 1 peut représenter l'identifiant du produit.
- Dans Express, on écrit :

```
app.get("/products/:id", (req, res) => {  
  const id = req.params.id;  
  res.send("Produit demandé : " + id);  
});
```

`http://localhost:3000/products`

`http://localhost:3000/products/1`

`http://localhost:3000/products/99`

`req.params.id` : récupère la valeur placée dans l'URL à la place de « id »

```
1 import express from "express";  
2  
3 const app = express();  
4  
5 const products = [  
6   { id: 1, name: "Ordinateur", price: 120000 },  
7   { id: 2, name: "Clavier", price: 3500 },  
8   { id: 3, name: "Souris", price: 2500 }  
9 ];  
10  
11 app.get("/", (req, res) => {  
12   res.send("<h1>Accueil</h1>");  
13 });  
14  
15 app.get("/products", (req, res) => {  
16   res.json(products);  
17 });  
18  
19 app.get("/products/:id", (req, res) => {  
20   const id = Number(req.params.id);  
21  
22   const product = products.find((item) => item.id === id);  
23  
24   if (!product) {  
25     return res.status(404).send("Produit introuvable");  
26   }  
27  
28   res.json(product);  
29 });  
30  
31 app.listen(3000, () => {  
32   console.log("Serveur lancé sur http://localhost:3000");  
33 });
```



■ Rappel :

JavaScript

```
const product = products.find((item) => item.id === id);
```

On peut la lire comme ceci :

Cherche dans le tableau `products` le premier produit dont `item.id` est égal à `id`.

`products` est un tableau d'objets

La méthode `.find()` sert à chercher un élément dans un tableau.

Elle parcourt le tableau élément par élément.

Exemple simple :

JavaScript

```
const numbers = [10, 20, 30];

const result = numbers.find((number) => number === 20);

console.log(result);
```

Résultat :

JavaScript

20

Donc `.find()` retourne le premier élément qui respecte la condition.

Cette partie est une fonction fléchée.

JavaScript

```
(item) => item.id === id
```

Elle veut dire :

JavaScript

```
function(item) {
  return item.id === id;
}
```



QUERY STRING

- La query string est la partie de l'URL après le ?.
- Exemple `:/search?keyword=node` Ici `:keyword=node` est une query string.
- Avec Express, on la récupère avec `:req.query`

Exemple

```
</> JavaScript
app.get("/search", (req, res) => {
  const keyword = req.query.keyword;

  res.send(`
    <h1>Recherche</h1>
    <p>Mot recherché : ${keyword}</p>
  `);
});
```

Tester

`http://localhost:3000/search?keyword=node`

Résultat :

Mot recherché : node

Exemple avec plusieurs paramètres

```
</> JavaScript
app.get("/filter", (req, res) => {
  const category = req.query.category;
  const maxPrice = req.query.maxPrice;

  res.send(`
    <h1>Filtre</h1>
    <p>Catégorie : ${category}</p>
    <p>Prix maximum : ${maxPrice}</p>
  `);
});
```

Tester :

`http://localhost:3000/filter?category=informatique&maxPrice=50000`



MÉTHODES HTTP : GET ET POST

- GET sert à demander une page ou des données.
- Dans Express :

```
app.get("/products", (req, res) => {  
  res.send("Liste des produits");  
});
```

- POST sert généralement à envoyer des données au serveur.

```
app.post("/products", (req, res) => {  
  res.send("Produit reçu");  
});
```

- Pour l'instant, on ne va pas utiliser de formulaire HTML.
Les formulaires seront vus dans le module **EJS + formulaires**.



MIDDLEWARES

- Un middleware est une fonction qui s'exécute entre la requête et la réponse.
- Schéma : requête → middleware → route → réponse
- Un middleware peut servir à :
 - afficher des logs ;
 - lire les données envoyées ;
 - servir des fichiers statiques ;
 - vérifier une condition ;
 - gérer les erreurs.

Ce middleware :

JavaScript

```
app.use((req, res, next) => {  
  console.log("Nouvelle requête :", req.method, req.url);  
  next();  
});
```

s'exécute à chaque requête.

La ligne :

JavaScript

```
next();
```

permet de continuer vers la route suivante.

Sans `next()`, la requête reste bloquée.

```
1 import express from "express";  
2  
3 const app = express();  
4  
5 app.use((req, res, next) => {  
6   console.log("Nouvelle requête :", req.method, req.url);  
7   next();  
8 });  
9  
10 app.get("/", (req, res) => {  
11   res.send("Accueil");  
12 });  
13  
14 app.get("/contact", (req, res) => {  
15   res.send("Contact");  
16 });  
17  
18 app.listen(3000, () => {  
19   console.log("Serveur lancé sur http://localhost:3000");  
20 });
```

Attention, dans le navigateur, certaines requêtes sont envoyées automatiquement. Nous, on a demandé /, mais Chrome peut aussi demander d'autres fichiers pour ses propres besoins. Ce n'est pas grave. Le middleware intercepte simplement tout ce qui arrive au serveur.



FICHIERS STATIQUES

- Les fichiers statiques sont les fichiers envoyés directement au navigateur.
- Exemples : CSS, images, JavaScript côté navigateur, icônes,
- Dans Express, on utilise :

```
app.use(express.static("public"));
```

Structure

```
module-4-express/  
|  
├─ app.js  
├─ package.json  
└─ public/  
    └─ css/  
        └─ style.css
```

Fichier public/css/style.css

```
</> CSS  
  
body {  
  font-family: Arial, sans-serif;  
  background-color: #f4f4f4;  
}  
  
h1 {  
  color: #333;  
}
```

```
import express from "express";  
  
const app = express();  
  
app.use(express.static("public"));  
  
app.get("/", (req, res) => {  
  res.send("Accueil");  
});  
  
app.listen(3000, () => {  
  console.log("Serveur lancé sur http://localhost:3000");  
});
```

Tester le fichier CSS

Ouvrir directement :

<http://localhost:3000/css/style.css>

Si le contenu du fichier CSS s'affiche, cela veut dire que le dossier public fonctionne.

On utilisera vraiment ce CSS dans le module EJS.

CONFIGURATION DE BASE D'UNE APPLICATION EXPRESS

Dans une application Express, on organise souvent `app.js` dans cet ordre :

1. importations
2. création de l'application
3. configuration
4. middlewares
5. routes
6. erreur 404
7. lancement du serveur

```
import express from "express";

const app = express();
const PORT = 3000;

// Configuration et middlewares
app.use(express.static("public"));

// Routes
app.get("/", (req, res) => {
  res.send("Accueil");
});

app.get("/about", (req, res) => {
  res.send("À propos");
});

app.get("/contact", (req, res) => {
  res.send("Contact");
});

// 404
app.use((req, res) => {
  res.status(404).send("Page introuvable");
});

// Lancement du serveur
app.listen(PORT, () => {
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
});
```



ORGANISATION ROUTES / CONTRÔLEURS

Quand une application grandit, il ne faut pas tout écrire dans `app.js`.

Au début, on écrit :

```
</> JavaScript

app.get("/products", (req, res) => {
  res.send("Liste des produits");
});
```

Mais si on a beaucoup de routes, `app.js` devient long.

On va donc séparer le code.

Structure

```
module-4-express/
|
├─ app.js
├─ package.json
|
├─ routes/
|   └─ productRoutes.js
|
└─ controllers/
    └─ productController.js
```

Rôle des fichiers

`app.js`

Il configure l'application et connecte les routes.

`routes/productRoutes.js`

Il contient les chemins liés aux produits.

Exemples :

```
/products
/products/search
/products/:id
```

`controllers/productController.js`

Il contient les fonctions qui répondent aux routes.

Exemples :

```
afficher tous les produits
afficher un produit
rechercher un produit
```



Créer un contrôleur

Créer le fichier :controllers/productController.js

Créer le fichier :routes/productRoutes.js

```
1 import express from "express";
2
3 import {
4   getAllProducts,
5   getProductById,
6   searchProducts
7 } from "../controllers/productController.js";
8
9 const router = express.Router();
10
11 router.get("/", getAllProducts);
12
13 router.get("/search", searchProducts);
14
15 router.get("/:id", getProductById);
16
17 export default router;
```

JavaScript

```
const router = express.Router();
```

permet de créer un groupe de routes.

Ici, on regroupe toutes les routes liées aux produits.

```
1 const products = [
2   { id: 1, name: "Ordinateur", price: 120000 },
3   { id: 2, name: "Clavier", price: 3500 },
4   { id: 3, name: "Souris", price: 2500 }
5 ];
6
7 export function getAllProducts(req, res) {
8   let html = "<h1>Liste des produits</h1>";
9
10   for (const product of products) {
11     html += `<p>${product.id} - ${product.name} : ${product.price} DA</p>`;
12   }
13
14   res.send(html);
15 }
16
17 export function getProductById(req, res) {
18   const id = Number(req.params.id);
19
20   const product = products.find((item) => item.id === id);
21
22   if (!product) {
23     return res.status(404).send("Produit introuvable");
24   }
25
26   res.send(`
27     <h1>Détail du produit</h1>
28     <p>Nom : ${product.name}</p>
29     <p>Prix : ${product.price} DA</p>
30   `);
31 }
32
33 export function searchProducts(req, res) {
34   const keyword = req.query.keyword;
35
36   if (!keyword) {
37     return res.send("Veuillez entrer un mot de recherche dans l'URL.");
38   }
39
40   const results = products.filter((product) =>
41     product.name.toLowerCase().includes(keyword.toLowerCase())
42   );
43
44   let html = `<h1>Recherche : ${keyword}</h1>`;
45
46   if (results.length === 0) {
47     html += "<p>Aucun produit trouvé.</p>";
48   }
49
50   for (const product of results) {
51     html += `<p>${product.name} - ${product.price} DA</p>`;
52   }
53
54   res.send(html);
55 }
```



`app.use("/products", productRoutes);`

signifie : Toutes les routes qui commencent par /products sont envoyées vers productRoute

`http://localhost:3000/`

`http://localhost:3000/about`

`http://localhost:3000/contact`

`http://localhost:3000/products`

`http://localhost:3000/products/1`

`http://localhost:3000/products/search?keyword=clavier`

`http://localhost:3000/test`

```
1 import express from "express";
2 import productRoutes from "../routes/productRoutes.js";
3
4 const app = express();
5 const PORT = 3000;
6
7 app.use(express.static("public"));
8
9 app.get("/", (req, res) => {
10   res.send("Bienvenue dans notre application Express");
11 });
12
13 app.get("/about", (req, res) => {
14   res.send("Application créée avec Node.js et Express");
15 });
16
17 app.get("/contact", (req, res) => {
18   res.send("Contact : contact@example.com");
19 });
20
21 app.use("/products", productRoutes);
22
23 app.use((req, res) => {
24   res.status(404).send("Page introuvable");
25 });
26
27 app.listen(PORT, () => {
28   console.log(`Serveur lancé sur http://localhost:${PORT}`);
29 });
```



